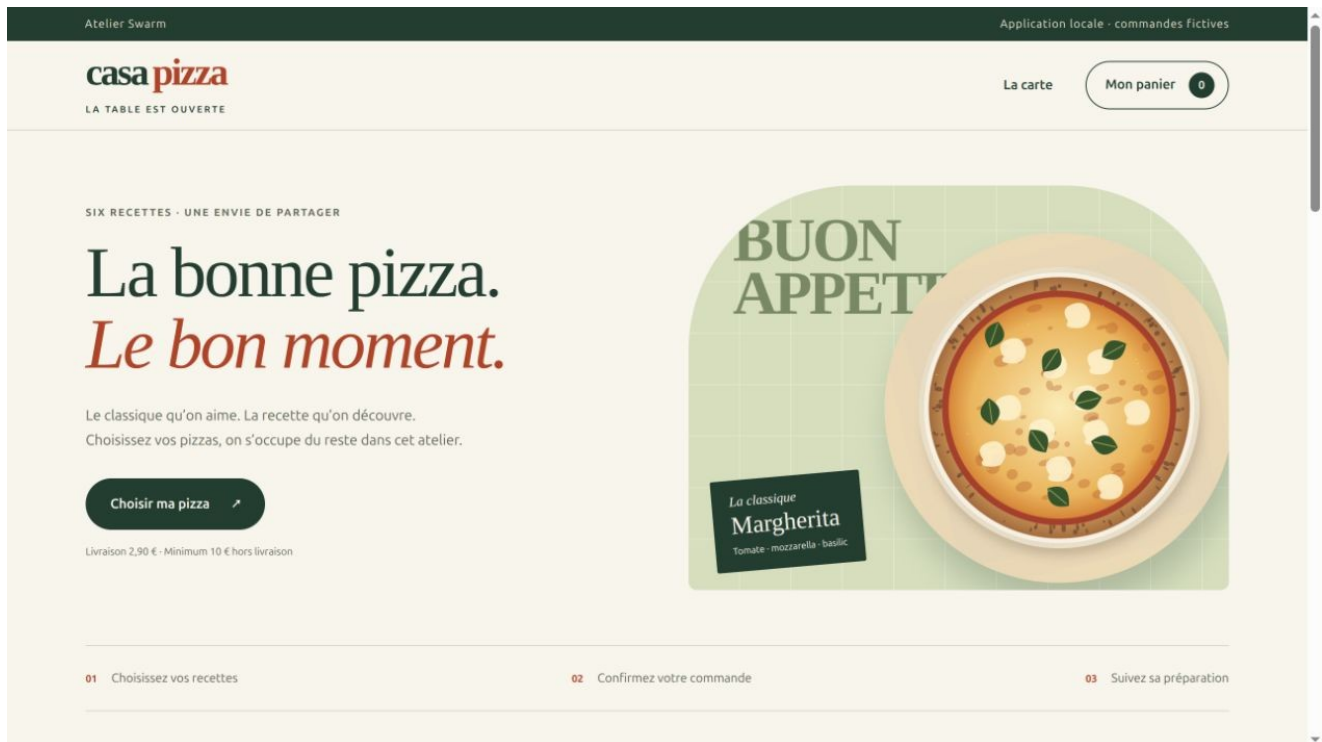


1 Guide de formation Swarm

Construire et vérifier une application de livraison de pizzas. Un atelier guidé pour comprendre les missions, les agents, les dépendances et les décisions humaines.



Casa Pizza est une application locale fonctionnelle avec six recettes et une commande persistante.

Le résultat attendu

Vous saurez préparer un besoin, lancer un exécutant, lire son activité, diagnostiquer un échec et vérifier une livraison. Vous jouerez ensuite le client et le restaurant pour tester une commande complète.

Public débutant. Édition du 9 octobre 2026. Les captures de production montrent un atelier à un exécutant et une recette du formateur. Les captures du plan à six tâches et des politiques montrent une démonstration de configuration, sans exécution de ces six tâches.

Les commandes utilisent Linux et bash. L'application de référence reste en français dans les deux éditions. Les nouvelles captures du cockpit suivent la langue du guide ; les captures historiques conservent leurs libellés d'origine.

2 Parcours de lecture

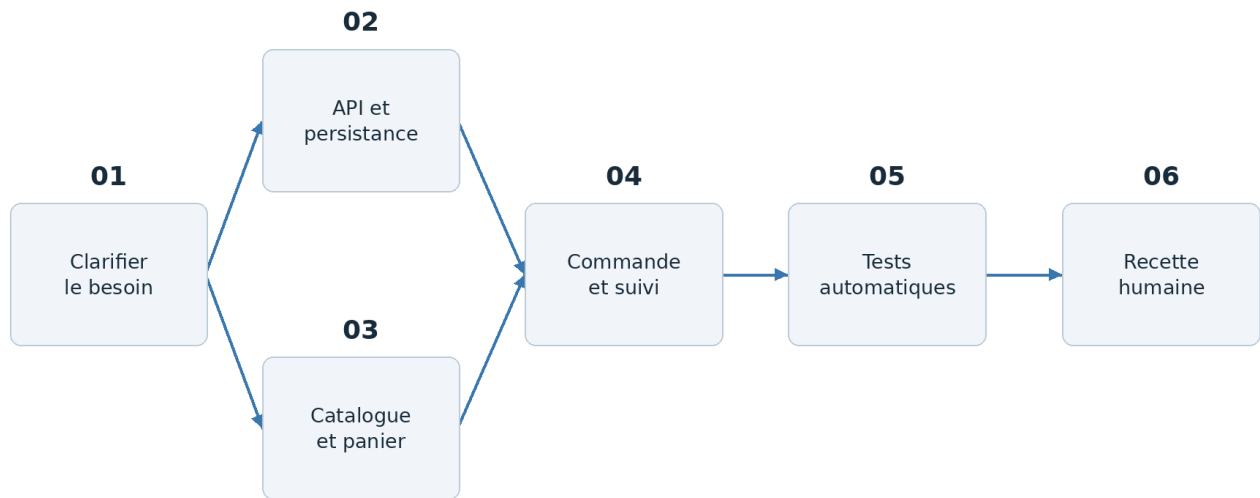
Suivez les chapitres dans l'ordre pour l'atelier complet. Pour un problème précis, utilisez les repères ci-dessous.

- 03 À quoi sert Swarm
- 04 Qui fait quoi
- 05 Préparer son poste
- 06 Essayer la référence locale
- 07 Créer son projet Swarm
- 08 Écrire un besoin précis
- 09 De la préparation à la réalisation
- 10 Lire la mission guidée
- 11 Lire le graphe des tâches
- 12 Lire une fiche de tâche
- 13 Explorer et modifier le graphe
- 14 Lancer une tâche délimitée
- 15 Suivre une exécution
- 16 Résoudre un démarrage en échec
- 17 Distinguer les formes de vérification
- 18 Configurer une revue humaine avec des contrôles
- 19 Définir le contrôle des tests
- 20 Examiner les conséquences avant confirmation
- 21 Comprendre la validation automatique
- 22 Vérifier que les preuves sont actuelles
- 23 Garder une personne dans la boucle
- 24 Comprendre une revue indépendante
- 25 Donner une direction au design
- 26 Vérifier la qualité graphique
- 27 Vérifier le catalogue et le panier
- 28 Tester une erreur utile
- 29 Confirmer et suivre une commande
- 30 Faire avancer le restaurant
- 31 Vérifier le suivi final et le mobile
- 32 Observer une commande sur mobile
- 33 Contrôler avant de valider
- 34 Diagnostiquer Failed to fetch
- 35 Refaire et expliquer le parcours
- 36 Périmètre et preuves de cet atelier

3 À quoi sert Swarm

Swarm est un cockpit local pour organiser et conduire le travail confié à des agents IA. Il relie le besoin, les tâches, les tentatives, les preuves et la décision de validation.

LE SENS DES DÉPENDANCES



La flèche signifie que le prérequis doit être validé avant la suite

Schéma pédagogique du plan • Ce schéma ne prouve aucune exécution

Ce schéma explique une organisation possible du travail. Il ne représente pas une exécution autonome réussie.

Un exemple concret

Pour Casa Pizza, le besoin dit ce que le client doit pouvoir faire. Une tâche demande du code et des tests. L'exécutant produit les fichiers. Swarm conserve l'activité et les tentatives ; le formateur teste ensuite la commande et décide si les critères sont satisfaits.

Le cockpit rend visibles les dépendances et les raisons d'attente. Il peut gérer des départs continus et des contrôles préautorisés lorsque l'organisation et les autorisations correspondantes sont configurées. La présence d'un graphe ne suffit pas à activer ces fonctions.

4 Qui fait quoi

Chaque acteur a une responsabilité distincte. La personne qui produit un résultat ne doit pas être confondue avec celle qui le vérifie ou l'accepte.

Acteur	Responsabilité	Dans cet atelier
Utilisateur	Définit le périmètre, autorise, examine et décide	Formateur et apprenant
Planificateur	Organise et délègue les tâches	Hiérarchie non configurée
Exécutant	Utilise ses outils pour produire les fichiers et son rapport	Un exécutant Codex
Moteur Swarm	Applique les conditions et contrôles autorisés	Historique et opérations publiques
Vérificateur indépendant	Examine un candidat dans une session distincte	Non utilisé pour la recette originale

Une IA de chat et un exécutant

Une connexion API peut aider à rédiger ou analyser un texte. Un exécutant équipé d'outils est nécessaire pour écrire les fichiers du projet. Configurer une connexion de chat ne prouve donc pas que cette IA peut fabriquer l'application.

Le nom d'une tâche, par exemple « Revue », ne configure pas à lui seul un vérificateur indépendant. Il faut examiner le rôle, la session, les autorisations et les preuves utilisées.

5 Préparer son poste

Ouvrez un terminal. C'est la fenêtre dans laquelle vous tapez les commandes ci-dessous, une ligne à la fois. Les commandes sont prévues pour Linux ou un terminal compatible bash.

Vérifier les programmes

```
python3 --version
git --version
codex --version
```

Python 3.10 ou plus récent lance Casa Pizza et ses tests. Git conserve un historique local. Codex doit déjà être installé et authentifié pour générer le code ; le kit ne réalise pas cette authentification.

Vérifier Swarm

Dans le dossier où vous avez installé le dépôt Swarm, vérifiez les deux fichiers : le lanceur `swarm.sh` et le binaire `bin/swarm`. Le formateur doit fournir une version fonctionnelle avant l'atelier.

```
ls -l swarm.sh bin/swarm
./swarm.sh --help
./bin/swarm --help
```

Comprendre les dossiers

Le dossier Swarm contient l'outil. Le dossier `pizza-atelier` contient le travail de l'élève. Le kit contient `projet-pizza`, la référence déjà terminée. Ne lancez pas une génération dans votre application habituelle.

Comprendre les adresses

127.0.0.1 désigne votre propre machine. Le cockpit Swarm et la webapp ont deux ports différents. Dans les captures : Swarm utilise 18842, Casa Pizza utilise 18841. Le lanceur Swarm utilise normalement 18792 si vous ne choisissez pas un autre port.

Cet atelier se déroule directement sur le poste. Si Swarm tourne en conteneur, le formateur doit d'abord préparer les chemins, les programmes agents et les accès dans ce conteneur.

Résultat attendu. Les trois programmes répondent et vous savez où se trouve votre installation Swarm.

6 Essayer la référence locale

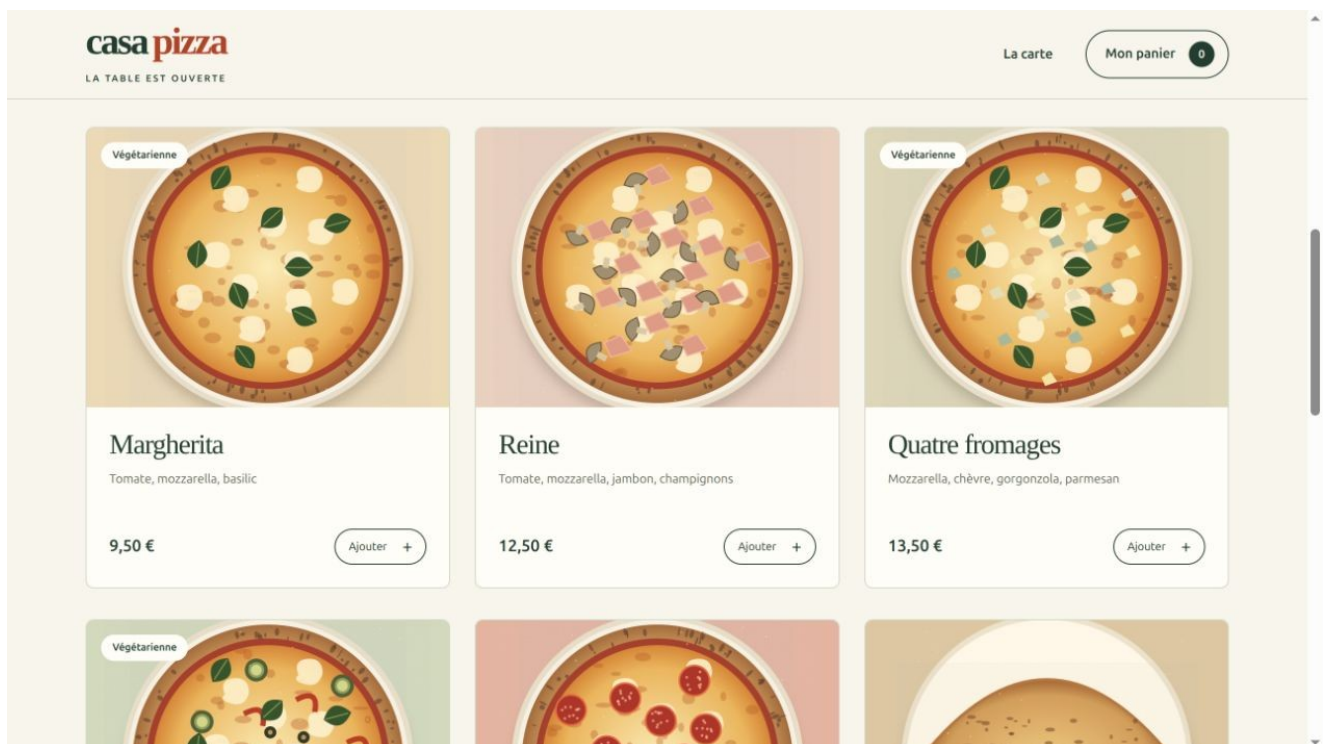
Commencez par découvrir ce qu'une livraison réussie doit permettre. Décompressez le kit et ouvrez un terminal dans le dossier extrait.

1. Entrez dans projet-pizza, le dossier du code de référence.
2. Lancez le serveur ; gardez ce terminal ouvert.
3. Ouvrez l'adresse ci-dessous dans votre navigateur.

```
cd projet-pizza  
python3 app.py
```

Adresse client : <http://127.0.0.1:18841/>

Adresse restaurant : <http://127.0.0.1:18841/restaurant>



Les boutons Ajouter alimentent le panier. Le catalogue utilise des illustrations locales, sans service externe.

Le mot de passe de démonstration du restaurant est atelier si aucune variable

CASA_PIZZA_RESTAURANT_PASSWORD n'est définie. Ce réglage est réservé à cet atelier local.

Résultat attendu. La carte s'affiche. Pour arrêter le serveur, revenez dans son terminal et faites Ctrl+C.

7 Créer son projet Swarm

Cette préparation crée une mission et une tâche dans un dossier neuf. Elle copie le besoin et les règles de travail, initialise Git et configure le programme Codex. Elle ne lance aucun agent et ne copie aucune clé.

1. Revenez dans le dossier extrait du kit, celui qui contient atelier_swarm.py.
2. Remplacez le chemin du binaire par celui de votre installation Swarm.
3. Choisissez un nouveau dossier pizza-atelier. Le script refuse un dossier déjà rempli.

```
python3 atelier_swarm.py \  
  --swarm /chemin/vers/swarm/bin/swarm \  
  --projet "$HOME/projets/pizza-atelier"
```

Le chemin /chemin/vers/swarm/bin/swarm est le seul chemin d'installation à adapter. Les guillemets autour du dossier projet permettent de conserver un chemin contenant des espaces.

Ouvrir le cockpit sur ce projet

Placez-vous dans votre dossier d'installation Swarm, puis lancez :

```
./swarm.sh start --root "$HOME/projets/pizza-atelier"
```

Ouvrez l'adresse locale indiquée par le lanceur. Ne partagez pas un lien de session contenant un accès privé.

Si un autre Swarm tourne déjà, utilisez un port libre :

```
./swarm.sh start --root "$HOME/projets/pizza-atelier" \  
  --address 127.0.0.1:18842
```

Réutilisez la même adresse lors des commandes status, open ou stop. Ne stoppez pas une mission appartenant à un autre projet pour libérer son port.

Résultat attendu. Le terminal affiche Projet prêt et un identifiant de mission. Dans Swarm, le travail Formation Casa Pizza est disponible.

8 Écrire un besoin précis

Avant d'envoyer une consigne, décrivez les actions du client et du restaurant. Une liste de comportements observables aide à vérifier la livraison.

► Préparation et méthode — reprendre un projet ou changer de méthode

Choisir un modèle de mission

Quel résultat souhaitez-vous obtenir ?

Aide : rédiger le besoin

Nom de la préparation

Casa Pizza pour un premier atelier

Votre besoin

Créer une webapp française de livraison de pizzas pour un atelier. Le client choisit parmi six pizzas, filtre les végétariennes, modifie son panier, renseigne une adresse fictive et confirme sa commande. Paiement à la livraison simulé. Le restaurant fait avancer le statut et le client suit sa commande. Les données persistent après redémarrage. Python 3 et SQLite, HTML CSS JS local, sans CDN ni package à installer. Vérifier le panier, les champs invalides, les montants recalculés par le serveur, les accès et le parcours mobile. Livrer code, tests, README et rapport. Aucun paiement réel, aucune adresse réelle et aucune publication Internet.]

Votre texte est enregistré dans ce projet. Aucun agent ne sera lancé.

Enregistrer le besoin Télécharger mon brouillon

Dans Préparer un projet, le champ **Votre besoin** précise le produit. **Enregistrer le besoin** sauvegarde le texte sans lancer un agent.

1. Ouvrez Préparer un projet si vous voulez enregistrer un brouillon personnel.
2. Donnez un nom et saisissez le texte ci-dessous.
3. Cliquez sur Enregistrer le besoin. Revenez ensuite à la mission Formation Casa Pizza créée par le kit.

Texte à copier

Créer une webapp française de livraison de pizzas pour un atelier. Le client choisit parmi six pizzas, filtre les végétariennes, modifie son panier, renseigne une adresse fictive et confirme sa commande. Paiement à la livraison simulé. Le restaurant fait avancer le statut et le client suit sa commande. Les données persistent après redémarrage. Python 3 et SQLite, HTML CSS JS local, sans CDN ni paquet à installer. Vérifier le panier, les champs invalides, les montants recalculés par le serveur, les accès et le parcours mobile. Livrer code, tests, README et rapport. Aucun paiement réel, aucune adresse réelle et aucune publication Internet.

Résultat attendu. Le besoin est enregistré. Le kit fournit aussi sa version détaillée dans docs/BESOIN.md ; la mission de cet atelier est créée par le script, sans adoption automatique du brouillon.

9 De la préparation à la réalisation

La préparation aide à clarifier le projet. Une proposition enregistrée reste différente d'un travail adopté et d'un agent lancé.

► Préparation et méthode — reprendre un projet ou changer de méthode

Choisir un modèle de mission

Quel résultat souhaitez-vous obtenir ?

Aide : rédiger le besoin

Nom de la préparation

Casa Pizza pour un premier atelier

Votre besoin

Créer une webapp française de livraison de pizzas pour un atelier. Le client choisit parmi six pizzas, filtre les végétariennes, modifie son panier, renseigne une adresse fictive et confirme sa commande. Paiement à la livraison simulé. Le restaurant fait avancer le statut et le client suit sa commande. Les données persistent après redémarrage. Python 3 et SQLite, HTML CSS JS local, sans CDN ni package à installer. Vérifier le panier, les champs invalides, les montants recalculés par le serveur, les accès et le parcours mobile. Livrer code, tests, README et rapport. Aucun paiement réel, aucune adresse réelle et aucune publication Internet.]

Votre texte est enregistré dans ce projet. Aucun agent ne sera lancé.

Enregistrer le besoin Télécharger mon brouillon

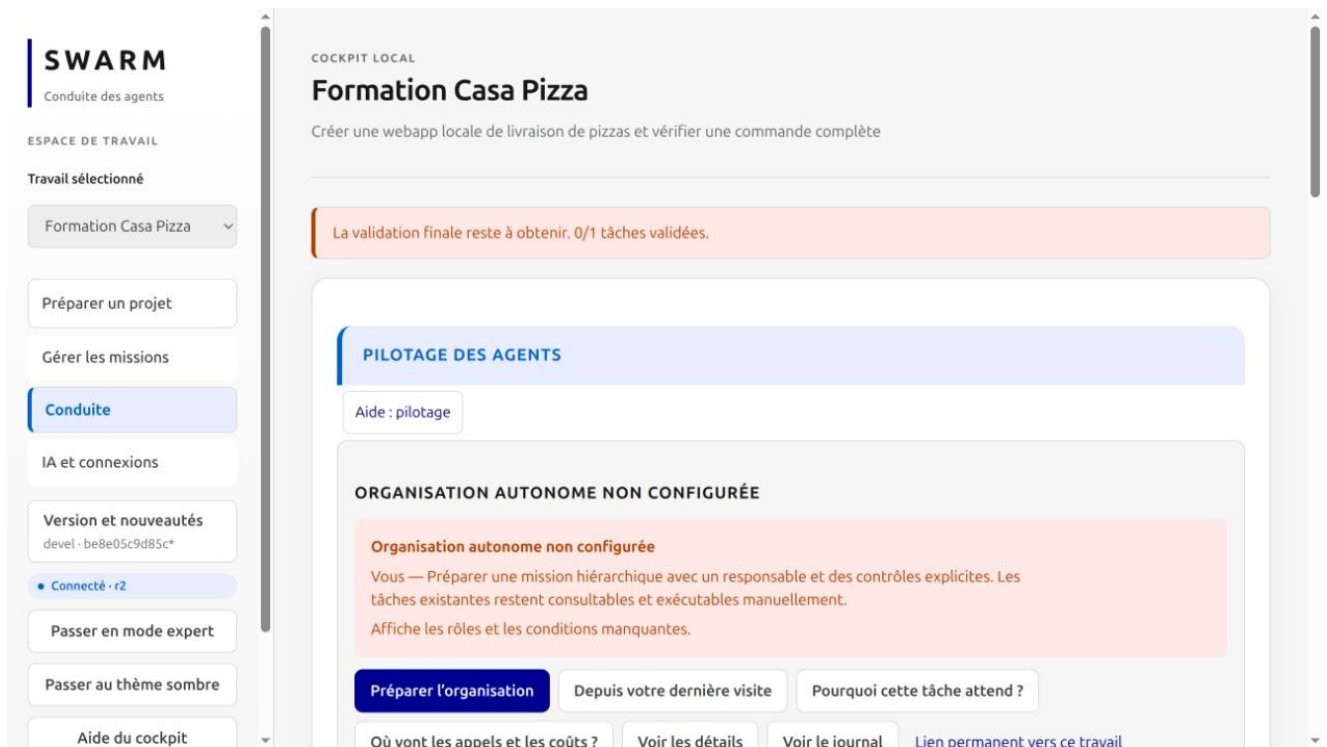
Le besoin est saisi dans Préparer un projet. Enregistrer conserve le texte ; ce bouton ne démarre pas un exécutant.

1. Décrire les utilisateurs, le problème, le résultat observable et les exclusions.
2. Relire le brief et les propositions de tâches avant leur adoption. Une réponse IA reste une proposition.
3. Vérifier ensuite dans Conduite que les tâches existent, avec critères et livrables.
4. Configurer l'exécutant et confirmer son lancement séparément.

Le kit fournit un raccourci reproductible : atelier_swarm.py crée un travail et une tâche à partir du besoin fourni. L'atelier ne prétend pas avoir exécuté toutes les étapes d'une préparation assistée ou d'une mission hiérarchique.

10 Lire la mission guidée

Choisissez Formation Casa Pizza dans Travail sélectionné, puis ouvrez Conduite. Relisez l'objectif avant de lancer le travail.



Le cockpit indique 0 sur 1 tâche validée. Organisation autonome non configurée concerne le pilotage hiérarchique.

1. Vérifiez que le travail sélectionné est Formation Casa Pizza.
2. Faites défiler la page jusqu'à la tâche Construire la webapp et ses tests.
3. Ouvrez la tâche et vérifiez son livrable : application, tests, README et rapport.

Pourquoi un message sur l'organisation

Cette mission est pilotée manuellement avec un exécutant. Le bandeau indique que le responsable et les contrôles d'une organisation autonome ne sont pas configurés. Les tâches restent exécutables manuellement, comme le précise le message. Ne présentez pas ce parcours comme une démonstration d'autonomie hiérarchique.

Agent ou connexion API

Une connexion API de discussion peut aider à rédiger un texte. Pour produire les fichiers du projet, il faut un exécutant disposant d'outils. Dans cet atelier, Codex est ce programme exécutant, configuré par le kit.

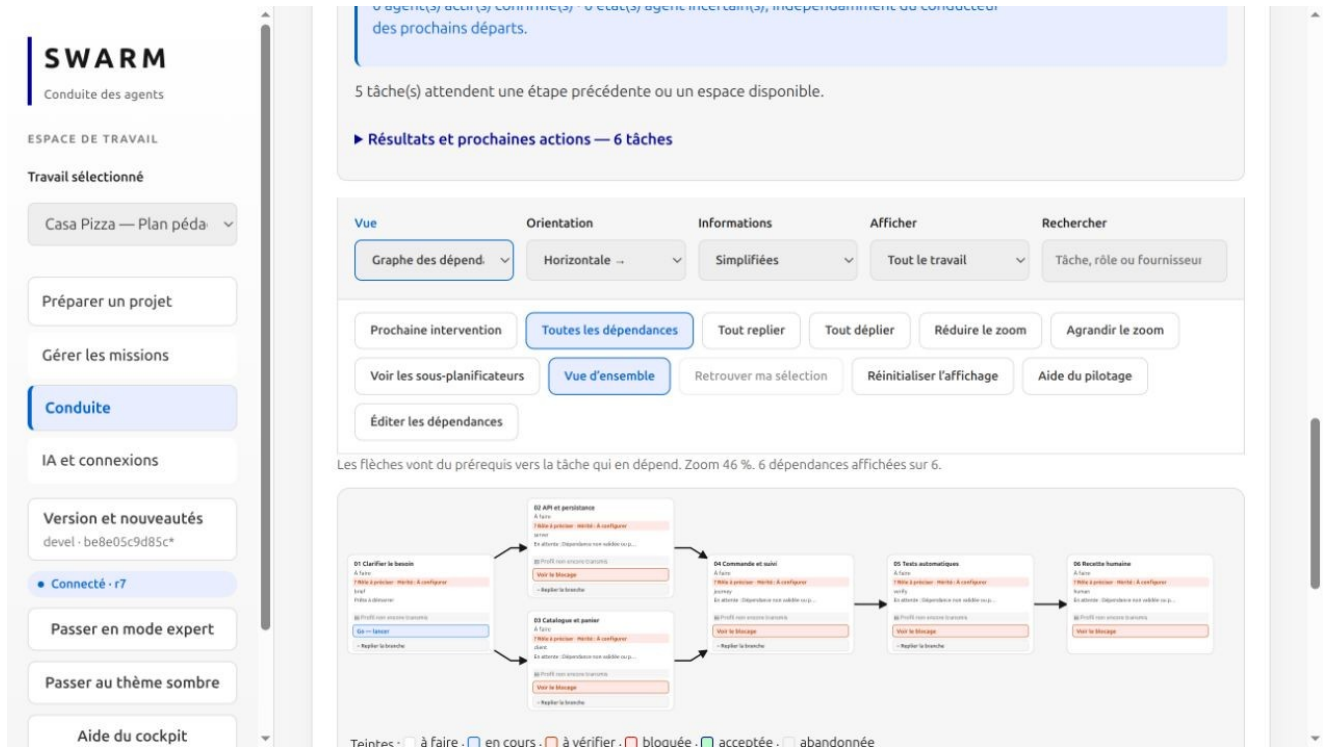
Résultat attendu. Vous identifiez une seule tâche et vous savez quel programme va produire ses fichiers.

11 Lire le graphe des tâches

Choisissez Casa Pizza — Plan pédagogique dans le sélecteur du travail. Il s'agit du plan de démonstration facultatif du kit, distinct de la mission originale à une tâche.

Pour créer ce plan facultatif, exécutez une seule fois la commande depuis le kit, après la création du projet. Elle ajoute un travail de démonstration sans modifier la mission originale ni lancer d'agent.

```
python3 creer_plan_pedagogique.py \
--swarm /chemin/vers/swarm/bin/swarm \
--projet "$HOME/projets/pizza-atelier"
```



Capture réelle du cockpit avec six tâches non exécutées. Les six flèches représentent leurs prérequis.

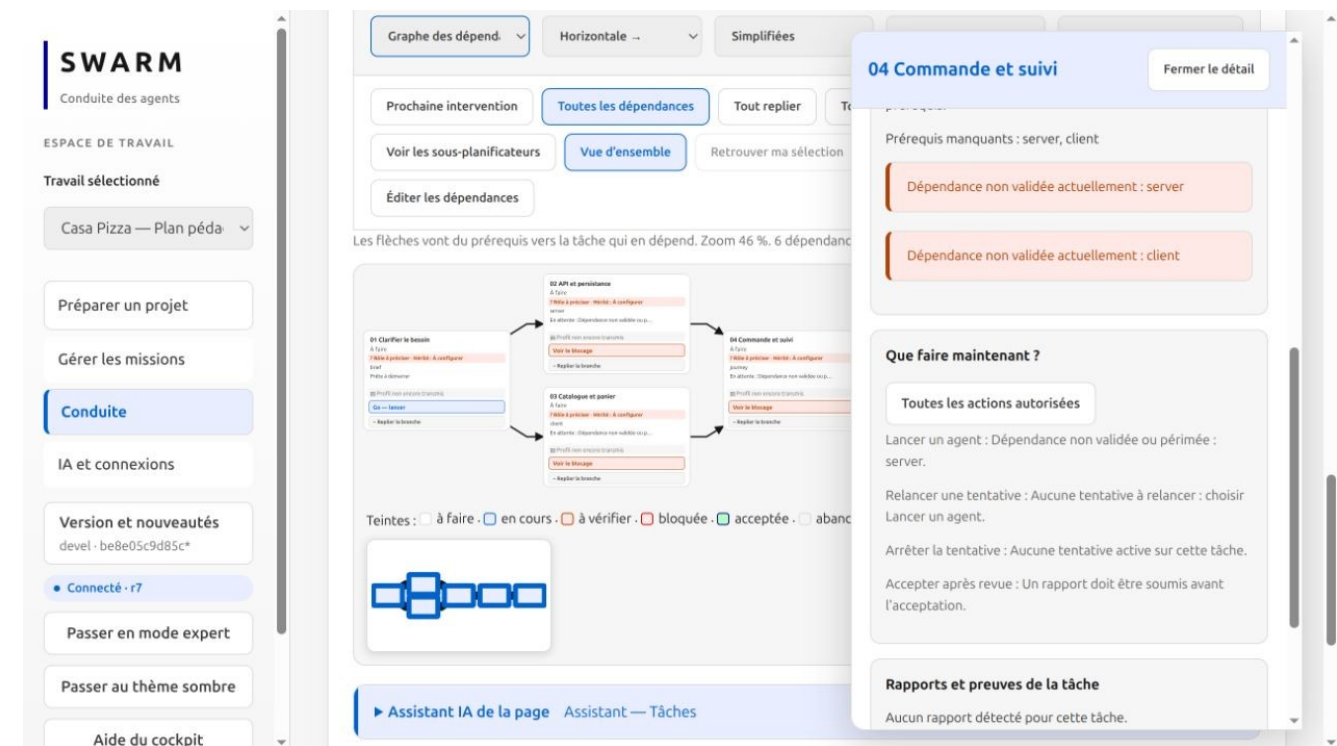
1. Repérer 01 Clarifier le besoin. Cette tâche n'a pas de prérequis.
2. Suivre les flèches vers 02 API et persistance et 03 Catalogue et panier.
3. Repérer la convergence sur 04 Commande et suivi : les deux prérequis sont nécessaires.
4. Lire la suite 05 Tests automatiques puis 06 Recette humaine.

Une flèche va du prérequis vers la tâche dépendante. Deux branches peuvent devenir éligibles après le même prérequis ; leur exécution simultanée dépend aussi des crénaux, budgets, rôles et espaces de travail disponibles.

Résultat attendu. Vous pouvez expliquer pourquoi Commande et suivi attend les deux branches. Aucune tâche de ce plan n'a été lancée.

12 Lire une fiche de tâche

Cliquez sur 04 Commande et suivi. La fiche rassemble l'état, les critères, les dépendances, les actions permises et les rapports.



La fiche décrit les prérequis manquants. Faites défiler le panneau pour lire Dépendances et impact et les actions autorisées.

Signal	Ce qu'il signifie
À faire	La tâche reste à réaliser ; cela ne signifie pas qu'elle peut démarrer.
Prête à démarrer	Les conditions observées permettent de préparer un départ.
En attente	Un prérequis, une preuve ou un espace manque. Lire la raison.
À vérifier	Un résultat demande une évaluation ; il n'est pas encore accepté.
Acceptée	Le résultat a une validation ; lire aussi l'historique du processus.

Distinguez toujours l'état de la tâche, l'état du processus et l'état de validation. Un processus terminé ou arrêté n'est pas une preuve de livraison correcte. Une tentative interrompue peut rester dans l'historique d'un résultat ensuite corrigé et accepté.

13 Explorer et modifier le graphe

Les commandes de vue facilitent la lecture. L'édition des dépendances modifie l'organisation du travail et exige une étape supplémentaire.

The screenshot shows the SWARM interface. On the left is a sidebar with the 'SWARM' logo and 'Conduite des agents' section. Below it, 'ESPACE DE TRAVAIL' contains 'Travail sélectionné' (Casa Pizza — Plan péda) and buttons for 'Préparer un projet', 'Gérer les missions', 'Conduite' (highlighted), 'IA et connexions', 'Version et nouveautés', 'Connecté · r7', 'Passer en mode expert', 'Passer au thème sombre', and 'Aide du cockpit'. The main area shows a status bar: '5 tâche(s) attendent une étape précédente ou un espace disponible.' and 'Résultats et prochaines actions — 6 tâches'. Below this is a toolbar with filters for 'Vue' (Graphes des dépend.), 'Orientation' (Horizontale), 'Informations' (Simplifiées), 'Afficher' (Tout le travail), and 'Rechercher' (Tâche, rôle ou fournisseur). Action buttons include 'Prochaine intervention', 'Toutes les dépendances', 'Tout replier', 'Tout déplier', 'Réduire le zoom', 'Agrandir le zoom', 'Voir les sous-planificateurs', 'Vue d'ensemble', 'Retrouver ma sélection', 'Réinitialiser l'affichage', and 'Aide du pilotage'. A text line states: 'Les flèches vont du prérequis vers la tâche qui en dépend. Zoom 46 %. 6 dépendances affichées sur 6.' The 'Éditer les dépendances' modal is open, showing a 'Quitter l'édition' button and instructions: 'Choisissez une opération, puis le prérequis et la tâche dépendante dans le graphe. Entrée et Espace offrent le même parcours que la souris.' It has three dropdowns: 'Opération' (Ajouter une dépendance), 'Prérequis' (Choisir une tâche), and 'Tâche dépendante' (Choisir une tâche). At the bottom are buttons for 'Ajouter au brouillon', 'Annuler', 'Rétablir', 'Prévisualiser', and 'Appliquer explicitement'. A status line says: '0 modification(s) dans le brouillon. La mission reste inchangée.' and a list item: '1. Aucune modification préparée.'

L'éditeur propose un prérequis, une tâche dépendante et un brouillon. Cette capture conserve zéro changement ; rien n'a été appliqué.

Explorer sans changer les tâches

Vue d'ensemble recadre le graphe. Le zoom permet de lire une carte. L'orientation change le sens de lecture. Les filtres affichent une partie du travail et la recherche aide à retrouver une tâche. Ces actions de présentation ne valident ni ne lancent de tâche.

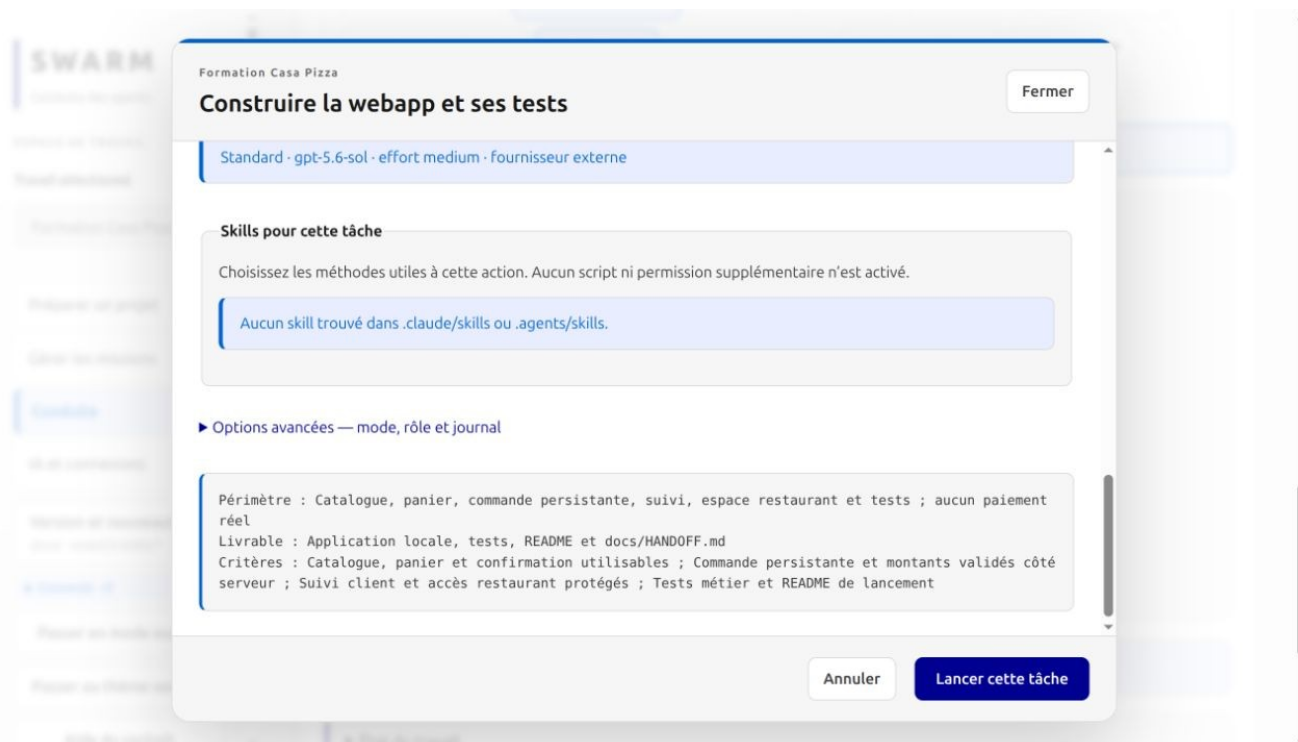
Comprendre une modification

Éditer les dépendances prépare un brouillon. L'opération indique Ajouter ou Retirer, puis le prérequis et la tâche dépendante. L'aperçu permet d'examiner l'effet avant Appliquer explicitement. Annuler et Rétablir concernent le brouillon, pas l'annulation d'un travail déjà exécuté.

Dans cet atelier, conservez les six dépendances du plan. Ne retirez pas un prérequis simplement pour faire disparaître une attente. Une branche masquée ou une couleur différente n'est pas une preuve que le travail correspondant est terminé.

14 Lancer une tâche délimitée

Ouvrez le lancement de la tâche. Relisez la consigne, le dossier et les critères avant de confirmer.



Le bas de la fenêtre rappelle le périmètre, le livrable et les critères. Lancer cette tâche démarre réellement l'exécutant.

1. Choisissez Codex parmi les exécutants disponibles. Vérifiez le dossier pizza-atelier.
2. Conservez les critères de la tâche. Aucun skill supplémentaire n'est nécessaire dans le kit.
3. Cliquez sur Lancer cette tâche, puis ouvrez Voir la session.

Consigne fournie par le kit

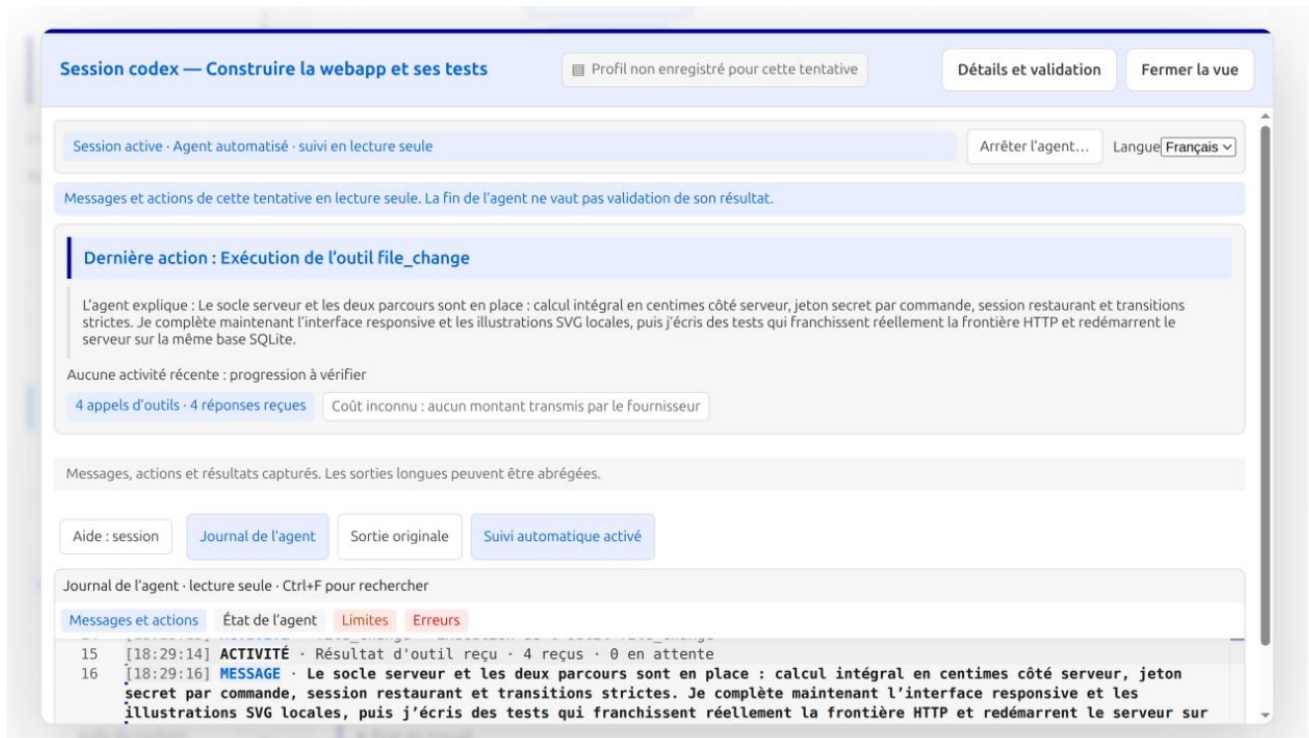
Réalise docs/BESOIN.md et respecte AGENTS.md. Livre code, tests, README et docs/HANDOFF.md. Python standard library, SQLite et ressources locales. Aucun autre agent et aucune publication. Ne lance pas de serveur durable. Vérifie montants, erreurs, accès et persistance. Ne modifie pas .swarm.

Le modèle affiché sur la capture est celui demandé lors de cet atelier. Il dépend de la configuration du fournisseur ; la capture ne prouve pas le modèle effectivement servi.

Résultat attendu. Une nouvelle tentative apparaît dans la session. Vous pouvez consulter ses messages et ses actions.

15 Suivre une exécution

Pendant la génération, utilisez la session pour distinguer messages, appels d'outils et erreurs. Le code produit doit encore être vérifié.



La session montre une action `file_change` et quatre réponses d'outils. Le message de l'agent décrit un travail en cours.

1. Lisez Dernière action pour voir ce que l'agent vient de faire.
2. Consultez Journal de l'agent, puis Messages et actions.
3. Si la progression semble arrêtée, regardez État de l'agent, Limites et Erreurs avant de relancer.

Aucune activité récente signifie que la progression doit être examinée. Coût inconnu signifie que le fournisseur n'a pas transmis de montant ; cela ne signifie pas que l'appel est gratuit.

Fin de session et résultat

Attendez le rapport de livraison. Vérifiez les fichiers créés et les tests réellement exécutés. Dans le cas observé, l'agent a produit le code puis s'est interrompu après des erreurs répétées de remise structurée. Le formateur a ensuite contrôlé et corrigé le résultat sur le poste.

Résultat attendu. Vous savez expliquer les dernières actions observées, sans confondre une déclaration de l'agent avec une preuve de fonctionnement.

16 Résoudre un démarrage en échec

La première tentative de cet atelier a échoué avant de produire du code. Sa sortie originale indiquait Not inside a trusted directory.

1. Ouvrez la session en échec, puis Sortie originale.
2. Lisez la première cause explicite ; ici, le dossier n'était pas un dépôt Git.
3. Vérifiez le dossier du projet. Le script du kit initialise déjà Git pour éviter ce problème.
4. Après correction, reprenez la même tâche afin de conserver l'historique de la tentative.

Vérification à faire dans le projet

```
git status
```

Si vous avez préparé le dossier manuellement et que Git indique qu'il ne s'agit pas d'un dépôt, le formateur peut initialiser ce dossier neuf avec `git init` puis enregistrer les fichiers de cadrage. N'initialisez pas au hasard un dossier contenant un autre projet.

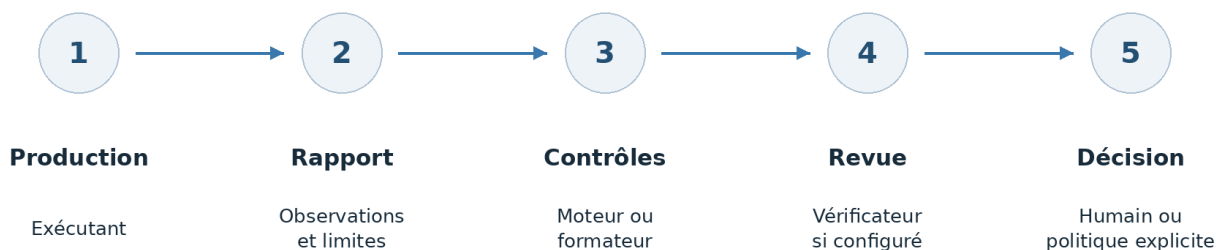
Une limite ou plusieurs erreurs identiques exigent un diagnostic. N'augmentez pas les budgets pour masquer un blocage. Si les sockets sont interdites dans le sandbox, exécutez les tests HTTP sur un poste local autorisé et consignez cette distinction.

Résultat attendu. La cause est identifiée, la correction est limitée au projet et l'échec reste visible dans l'historique.

17 Distinguer les formes de vérification

L'expression « auto vérification » peut désigner des opérations différentes. Identifiez l'acteur et la preuve avant de conclure.

DE LA PRODUCTION À LA DÉCISION



Chaque étape apporte une information différente sur le résultat

Schéma explicatif • Les contrôles seuls ne couvrent pas un jugement qualitatif

Les observations du producteur, les contrôles et la décision ont des rôles différents.

Étape	Preuve attendue	Limite
Tests du producteur	Commande, sortie et résultat réels	Pas une revue indépendante
Contrôles du moteur	Reçu sur le candidat et les fichiers déclarés	Couvre les critères testables déclarés
Revue indépendante	Verdict dans une session distincte	Jugement du vérificateur à examiner
Recette humaine	Parcours observé et rapport	Périmètre réellement testé uniquement

Dans la mission originale, le producteur a exécuté quatre tests métier. Le formateur a ajouté sept tests HTTP, exécuté les onze tests et rejoué le parcours navigateur. Ces contrôles n'ont pas été lancés par une politique automatique Swarm.

18 Configurer une revue humaine avec des contrôles

L'exemple suivant montre un aperçu de configuration sur la tâche 05 Tests automatiques. Il n'autorise aucune mission continue et ne lance aucun agent.

The screenshot shows a modal window titled 'Configurer les validations — 05 Tests automatiques' with a 'Fermer' button in the top right. The window is set against a background of the SWARM interface, which includes a sidebar with 'Conduite' selected and a main area with a task list. Inside the modal, the text 'Choisissez ce qui reste en revue humaine ou préautorisez uniquement des contrôles structurés.' is followed by a 'Comprendre cette action' button. A 'Mode de validation' dropdown menu is set to 'Revue humaine'. A blue information box states: 'Portée : cette tâche seulement. Une suggestion IA ne sera jamais convertie en autorisation. Si un critère exige un jugement qualitatif, conservez la revue humaine.' Below this, a note says 'En revue humaine, les contrôles facultatifs produisent des preuves sans accepter la tâche.' At the bottom, there is a 'Critères à couvrir' section and two buttons: 'Annuler' and 'Examiner l'effet'.

Le mode humain conserve la décision d'acceptation. Les contrôles facultatifs peuvent apporter des preuves.

1. Dans Conduite, cliquez sur Voir les détails pour ouvrir Résultats et prochaines actions.
2. Sur 05 Tests automatiques, choisissez Configurer les validations. Le bouton de gate dans la fiche de tâche est une autre opération.
3. Gardez Revue humaine. Lisez les critères avant d'ajouter un contrôle.
4. Ajoutez uniquement une commande réellement disponible dans l'environnement qui exécutera Swarm.

Si un critère demande un jugement qualitatif, conservez le mode humain pour toute la tâche. Les commandes peuvent soutenir la revue sans remplacer la personne. Une suggestion IA n'est jamais, à elle seule, une autorisation.

19 Définir le contrôle des tests

Cet exemple utilise la suite de la référence Casa Pizza. Sur votre propre candidat, vérifiez d'abord les fichiers, le nombre de tests et la commande applicable.

The screenshot shows a modal window titled 'Configurer les validations — 05 Tests automatiques' with a 'Fermer' button in the top right. The window contains the following fields:

- Fichiers :** app.py, tests/test_app.py, tests/test_http.py, static/app.js, static/restaurant.js
- Identifiant :** pizza-tests
- Programme :** python3 (dropdown menu)
- Arguments — un argument exact par ligne :** -W, error::ResourceWarning, -m, unittest, -v
- Justification objective de la couverture :** La suite du candidat vérifie calcul des prix, erreurs, accès, transitions et persistance. Un échec de test renvoie un code non nul.

At the bottom, there are two buttons: 'Annuler' and 'Confirmer cet effet'.

Le programme et les arguments sont séparés. Aucun shell implicite ne transforme les lignes en script.

Identifiant : pizza-tests. Programme : python3. Répertoire : . (racine du projet). Délai : 60 secondes.
Arguments, un par ligne :

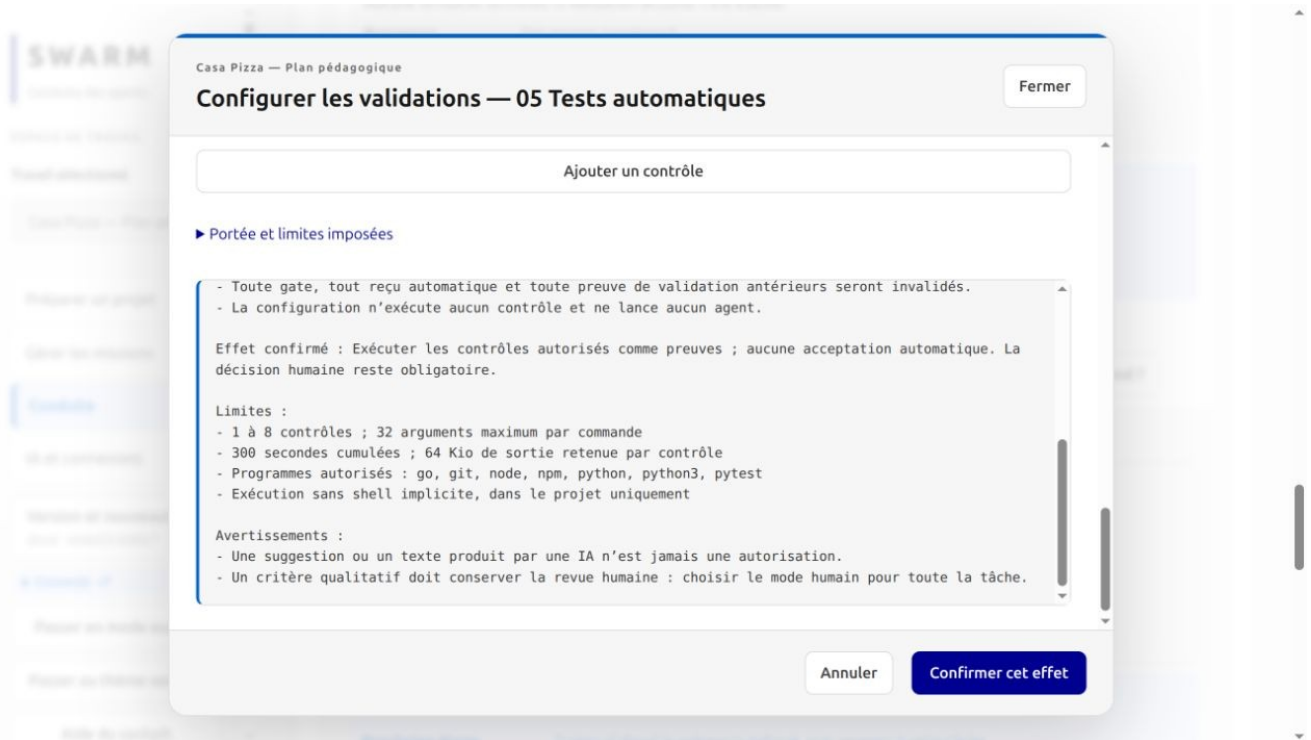
```
-W
error::ResourceWarning
-m
unittest
-v
```

Cochez seulement le critère « Les onze tests du candidat réussissent ». Justifiez la couverture par les comportements exercés par ces tests. Ce contrôle ne démontre pas à lui seul l'ergonomie, l'accessibilité complète ou une aptitude à la production.

Fichiers examinés : app.py, tests/test_app.py, tests/test_http.py et les fichiers HTML, JS et CSS utilisés. Les chemins sont relatifs à la racine du projet, même si le répertoire de commande est différent. Ajoutez aussi tout autre fichier dont votre verdict dépend.

20 Examiner les conséquences avant confirmation

Cliquez sur Examiner l'effet. L'aperçu est une occasion de vérifier la portée exacte avant d'enregistrer la politique.



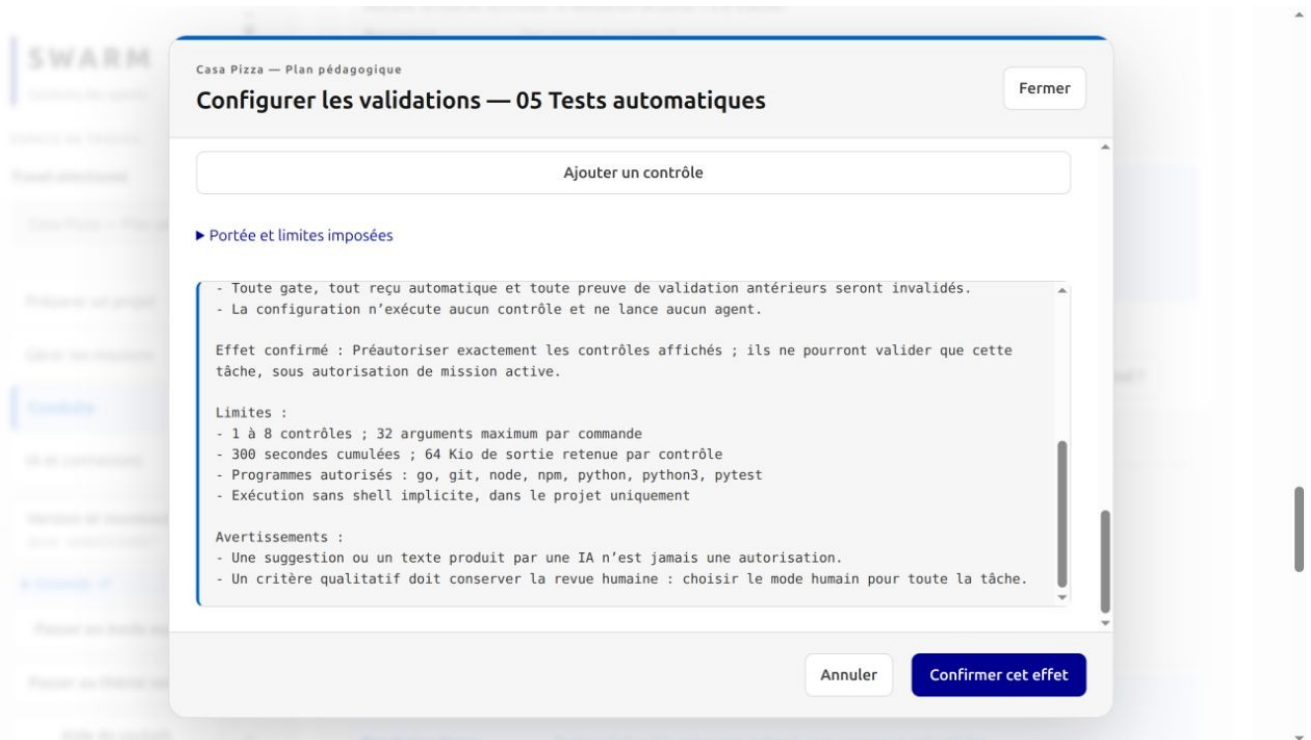
Aperçu du mode humain. La configuration ne lance ni contrôle ni agent et l'acceptation automatique reste exclue.

1. Relire l'identifiant de tâche, la commande, les fichiers, les critères, le répertoire et le délai.
2. Vérifier que la décision humaine reste obligatoire.
3. Lire l'invalidation des anciennes gates et preuves. Une politique modifiée exige une nouvelle évaluation applicable au candidat.
4. Confirmer seulement si cet effet est voulu. Pour la démonstration du guide, Annuler a été choisi.

Partager la sortie avec le vérificateur est facultatif et limité à 8 Kio. Les sorties peuvent contenir des données sensibles ; l'exemple conserve cette option décochée. Déclarer des fichiers incomplets réduit ce que la preuve peut établir.

21 Comprendre la validation automatique

Le mode automatique concerne des critères objectifs entièrement couverts par des contrôles explicitement préautorisés. L'aperçu ci-dessous n'a pas été confirmé.



Aperçu réel du mode automatique. Sa portée reste cette tâche, sous autorisation de mission active.

Enregistrer une politique ne lance aucun contrôle. Le moteur exige notamment une mission continue autorisée, non en pause, la bonne tentative de production terminée et un résultat soumis. Les critères doivent être couverts ; les programmes et délais sont bornés.

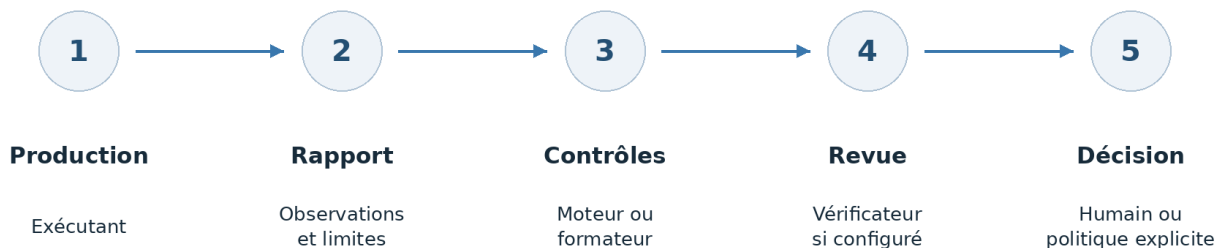
Un contrôle échoué empêche l'acceptation. Une réussite peut laisser une revue indépendante requise. En mode humain, elle laisse la décision humaine obligatoire. Le passage au vert dépend de l'ensemble des gardes applicables, pas simplement d'un message « OK ».

Le moteur autorise 1 à 8 contrôles, 32 arguments au maximum par commande, 300 secondes cumulées et jusqu'à 64 Kio de sortie retenue par contrôle. Cette configuration est montrée pour comprendre le mécanisme ; la mission originale reste un atelier guidé.

22 Vérifier que les preuves sont actuelles

Une réussite concerne un candidat précis. Elle ne couvre pas automatiquement un fichier modifié après le contrôle.

DE LA PRODUCTION À LA DÉCISION



Chaque étape apporte une information différente sur le résultat

Schéma explicatif • Les contrôles seuls ne couvrent pas un jugement qualitatif

L'acceptation doit porter sur le candidat effectivement contrôlé et revu.

1. Identifier la tâche, la tentative et le rapport concernés.
2. Consulter la commande, le code de retour, les erreurs et les limites de sortie.
3. Vérifier les fichiers déclarés et leurs empreintes. Sans fichiers déclarés, un reçu lié au seul rapport ne prouve pas la stabilité de tout le code.
4. Après correction du code ou de la politique, refaire les contrôles nécessaires et la revue applicable.

Un reçu actuel peut être réutilisé sans relancer une commande à chaque passage du conducteur. L'historique reste conservé lorsque les preuves deviennent périmées. Un contrôle qui n'a pas pu s'exécuter n'établit pas un défaut métier du code : diagnostiquer d'abord son environnement.

Exemple

Les onze tests passent. Vous modifiez ensuite app.py. Le résultat précédent ne suffit plus pour ce nouveau candidat. Relancez la suite et le parcours affecté, puis mettez à jour le rapport avant de demander l'acceptation.

23 Garder une personne dans la boucle

La personne intervient aux moments où une décision change le périmètre, l'exécution ou l'acceptation. Elle a besoin d'un résultat concret et de preuves lisibles.

Moment	À examiner	Décision humaine
Avant le plan	Besoin, exclusions et critères observables	Adopter ou corriger
Avant le départ	Outils, dossier, instruction, limites et modèle demandé	Autoriser le départ prévu
Avant une politique	Commandes exactes, fichiers, portée et invalidation	Confirmer ou annuler
Après livraison	Tests réels, parcours, rapport et réserves	Accepter ou demander correction
Après un blocage	Cause, tentative concernée, effet de la reprise	Reprendre de façon ciblée

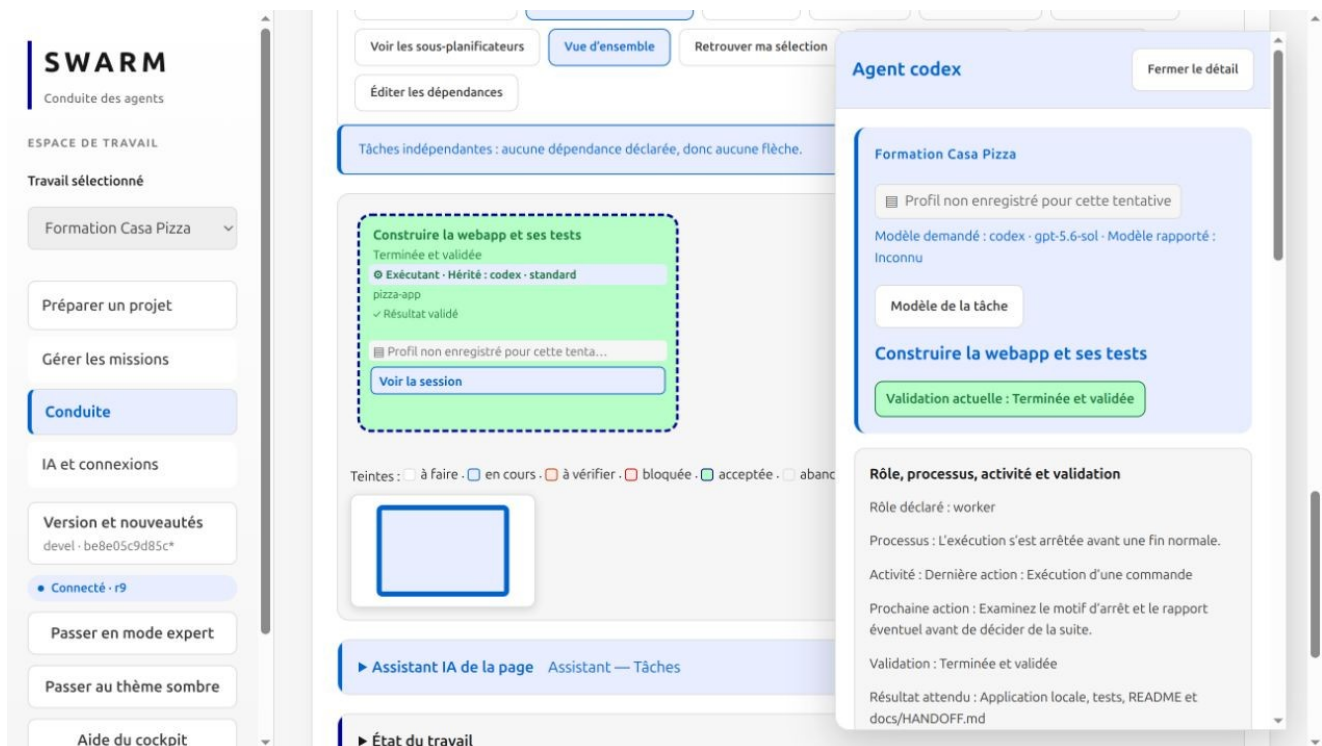
Questions de recette pour Casa Pizza

Le total est-il compréhensible ? Les erreurs indiquent-elles comment corriger la commande ? Les boutons sont-ils utilisables au clavier ? Le suivi montre-t-il la même commande que le restaurant ? Le parcours reste-t-il lisible à 390 pixels ? Les fonctions hors périmètre sont-elles annoncées ?

Consignez l'observation et la limite. « Cela fonctionne » est trop vague. Préférez « deux Margherita et une Reine, total 34,40 euros, même numéro côté restaurant, statut Livrée visible après actualisation ».

24 Comprendre une revue indépendante

Une revue indépendante apporte un second examen du candidat. Elle demande une configuration effective et ne se déduit pas du nom d'une tâche.



L'atelier original conserve une validation du formateur et un processus interrompu. Cette capture ne prouve aucune revue indépendante.

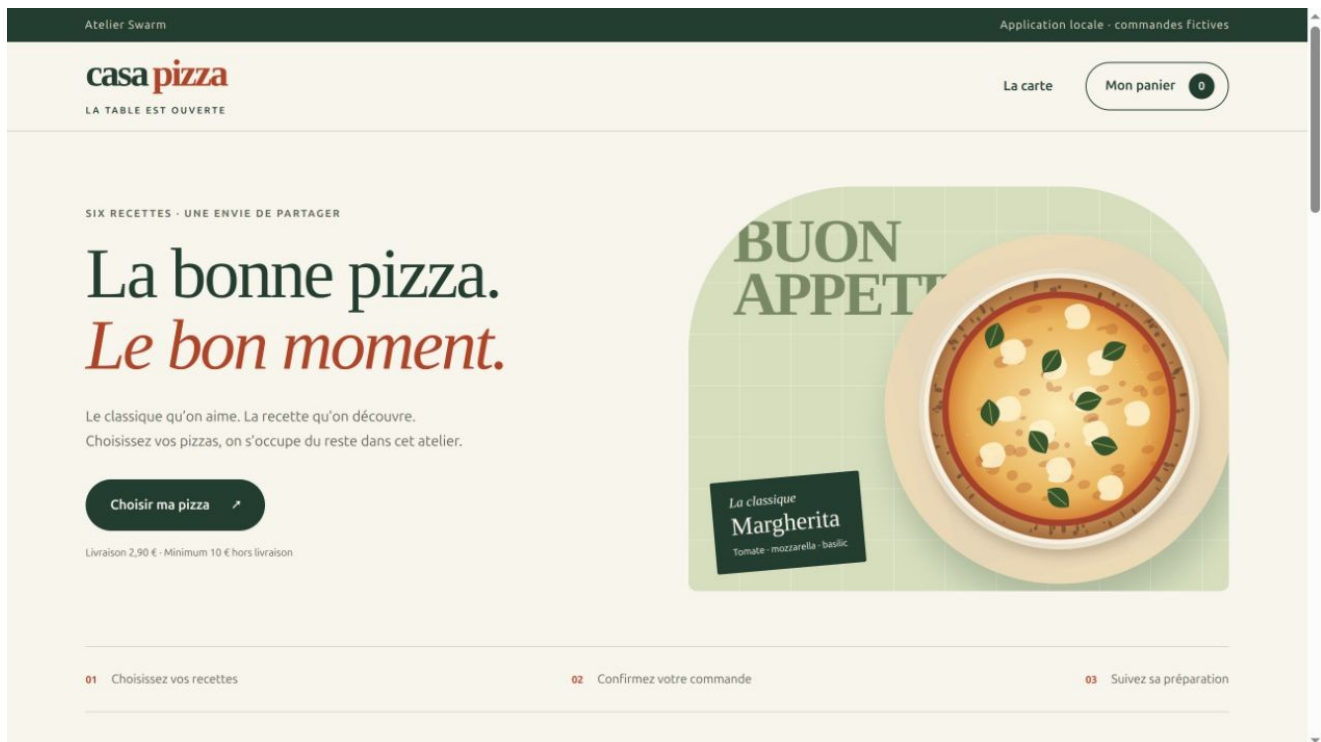
Le vérificateur doit recevoir le rapport et les preuves pertinentes dans une session distincte, avec un périmètre et des limites. Son verdict doit être lié au candidat examiné. Il ne doit pas produire discrètement des corrections puis juger son propre résultat.

Un verdict favorable ne remplace pas un contrôle obligatoire échoué. Un refus exige une correction ciblée et une nouvelle évaluation. Une preuve devenue périmée exige une revue applicable au candidat actuel.

Pour ce guide, la recette originale a été faite par le formateur dans la session de réalisation. Elle est explicitement décrite comme telle. Les schémas expliquent les responsabilités possibles ; ils ne prétendent pas démontrer une hiérarchie autonome ou une revue indépendante exécutée.

25 Donner une direction au design

Un besoin fonctionnel ne suffit pas à définir un rendu de qualité. Le premier brief précisait des couleurs et des cartes, mais la première composition restait trop générique pour une présentation publique.



L'accueil de référence a été retravaillé dans la session du formateur après l'atelier initial. Ce rendu illustré utilise uniquement des ressources locales.

Une instruction plus précise

Créer une trattoria illustrée : papier crème, encre vert sombre, accent terre cuite, titres serif et texte utilitaire sobre. Composer un accueil original avec une illustration locale détaillée et un bouton clair vers la carte. Différencier les six recettes. Reprendre cette identité dans le panier et le suivi. Montrer les montants et les erreurs avec netteté. Vérifier la version mobile et le focus clavier. Ne pas inventer de notes clients, de provenance ni de délai de livraison.

Les images de la référence sont des illustrations SVG, pas des photographies. Le choix de ressources locales conserve un atelier sans téléchargement externe. Une autre direction graphique exige un brief et une recette adaptés.

26 Vérifier la qualité graphique

La qualité graphique demande une décision humaine sur le vrai rendu. Des tests métier réussis ne rendent pas automatiquement une page agréable ou distinctive.

casa pizza
LA TABLE EST OUVERTE

La carte Mon suivi **Mon panier 3**

Votre commande.

Une dernière vérification avant de confirmer.

Votre sélection

01 / PANIER

Margherita
9,50 € pièce

Quantité

Reine
12,50 € pièce

Quantité

Sous-total 31,50 €

Livraison 2,90 €

Total 34,40 €

Minimum de commande : 10,00 € hors livraison.

Où vous livrer ?

02 / COORDONNÉES

Pour cet atelier, utilisez uniquement des coordonnées fictives.

Nom

Adresse fictive

Téléphone fictif

Confirmer la commande

Paiement à la livraison simulé - aucun paiement réel

Le panier de référence sépare sélection, montants et coordonnées. La revue vérifie la lisibilité et le parcours, au-delà du simple clic fonctionnel.

Point à examiner	Action du relecteur
Hiérarchie	Trouver l'action principale et le prix sans chercher
Cohérence	Comparer accueil, carte, panier et suivi
Mobile et clavier	Tester à 390 px et suivre le focus visible
Vérité du produit	Vérifier les messages, les montants et les promesses

La refonte actuelle et sa nouvelle recette sont consignées dans projet-pizza/docs/REFONTE.md. Les captures d'acceptation Swarm montrent la version historique. Elles ne constituent pas une validation de cette révision. Le formateur a repris le parcours dans le navigateur et les onze tests ; aucune revue indépendante de la refonte n'est revendiquée.

27 Vérifier le catalogue et le panier

Une fois le code disponible, ouvrez un second terminal dans votre dossier pizza-atelier. Lancez python3 app.py, puis ouvrez <http://127.0.0.1:18841/>. Arrêtez auparavant la référence si elle occupe ce port.

casa pizza
LA TABLE EST OUVERTE

La carte Mon suivi **Mon panier 3**

Votre commande.

Une dernière vérification avant de confirmer.

Votre sélection 01 / PANIER

	Margherita 9,50 € pièce	Quantité 2
	Reine 12,50 € pièce	Quantité 1
Sous-total		31,50 €
Livraison		2,90 €
Total		34,40 €

Minimum de commande : 10,00 € hors livraison.

Où vous livrer ? 02 / COORDONNÉES

Pour cet atelier, utilisez uniquement des coordonnées fictives.

Nom

Adresse fictive

Téléphone fictif

Confirmer la commande

Paiement à la livraison simulé - aucun paiement réel

Deux Margherita et une Reine donnent 31,50 euros de produits plus 2,90 euros de livraison, soit 34,40 euros.

1. Vérifiez les six recettes. Cochez Végétariennes uniquement : seules Margherita, Quatre fromages et Végétarienne doivent rester.

2. Décochez le filtre. Ajoutez une Margherita et une Reine.

3. Ouvrez Mon panier, puis passez la quantité de Margherita à 2. Au clavier, utilisez les flèches du champ de quantité.

4. Saisissez Camille Exemple, 12 rue de la Démonstration et 060000000000.

Résultat attendu. Le panier totalise 34,40 euros. Les quantités et les champs sont accessibles au clavier.

Les valeurs sont contrôlées par le serveur. Le navigateur affiche un calcul pratique, mais il ne doit pas pouvoir imposer un prix falsifié.

28 Tester une erreur utile

Avant de confirmer un panier valide, vérifiez que le produit explique un refus. Cette étape permet de tester la règle de minimum.

The screenshot shows the Casa Pizza checkout interface. On the left, the cart contains one Margherita pizza for 9.50 €. The subtotal is 9.50 €, and the delivery fee is 2.90 €, resulting in a total of 12.40 €. A note indicates a minimum order of 10.00 € for delivery. On the right, the checkout form is filled with fictitious data: Name (Camille Exemple), Address (12 rue de la Démonstration), and Phone (0600000000). A red error message states: 'Le minimum hors livraison est de 10,00 €.' The 'Confirmer la commande' button is visible, and a footer note mentions 'Paieement à la livraison simulé - aucun paieement réel'.

Une seule Margherita vaut 9,50 euros hors livraison. Le total de 12,40 euros ne suffit donc pas au minimum de produits.

1. Sur un panier neuf, ajoutez uniquement une Margherita.
2. Renseignez les coordonnées fictives, puis cliquez sur Confirmer la commande.
3. Vérifiez le message Le minimum hors livraison est de 10,00 euros.
4. Ajoutez une pizza ou augmentez la quantité : le message doit disparaître lorsque le panier est corrigé.

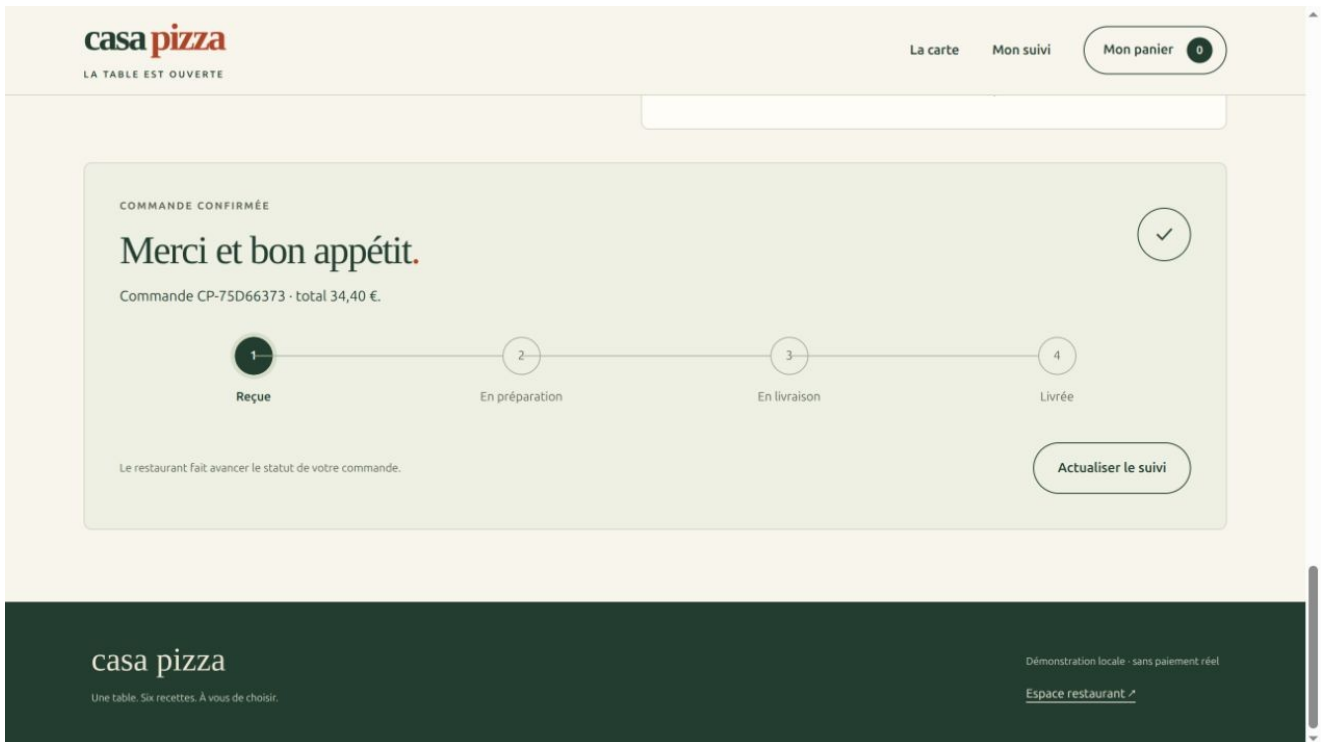
Résultat attendu. Le serveur refuse la commande insuffisante. La règle porte sur les produits, pas sur les frais de livraison.

Deux autres contrôles

Essayez un nom vide et une quantité invalide. La commande doit rester bloquée avec une erreur compréhensible. Réparez ensuite les champs pour poursuivre le scénario valide.

29 Confirmer et suivre une commande

Reconstituez le panier de deux Margherita et une Reine. Une confirmation doit donner un numéro, le montant et un statut initial.



La commande de référence CP 75D66373 a été reçue pour 34,40 euros. Votre numéro sera différent.

1. Cliquez une seule fois sur Confirmer la commande.
2. Notez le numéro affiché et vérifiez le total de 34,40 euros.
3. Gardez cet onglet ouvert pour conserver le suivi individuel.
4. Utilisez Actualiser le suivi après un changement côté restaurant.

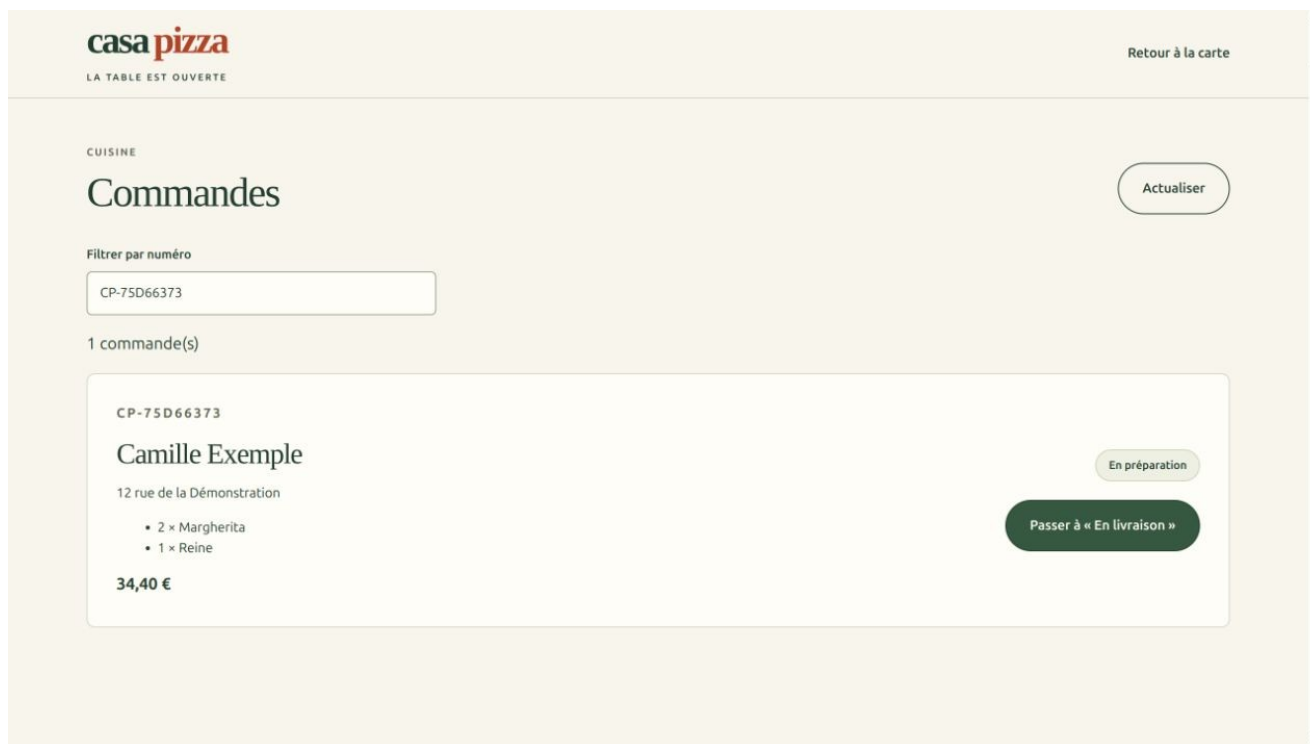
Résultat attendu. Le premier statut est Reçue. Un numéro visible et un montant correct attestent la création de cette commande.

Le navigateur garde un jeton privé de suivi dans sa session. Le numéro seul ne permet pas à un autre navigateur de lire la commande. Ne copiez pas des jetons ni de données réelles dans un rapport de formation.

Le serveur conserve la commande dans SQLite. Un redémarrage sur la même base conserve les commandes, mais il invalide les sessions restaurant conservées en mémoire.

30 Faire avancer le restaurant

Ouvrez <http://127.0.0.1:18841/restaurant> dans un deuxième onglet. Vous jouez maintenant le rôle de la cuisine.



La même commande contient deux Margherita et une Reine. Elle est en préparation ; le bouton propose la livraison.

1. Essayez une fois un mauvais mot de passe : l'accès doit être refusé.
2. Connectez-vous avec atelier si vous utilisez la configuration locale de démonstration.
3. Repérez votre numéro de commande, éventuellement avec Filtrer par numéro. Faites-le passer de Reçue à En préparation.
4. Cliquez ensuite pour passer à En livraison, puis à Livrée.

Résultat attendu. Le restaurant affiche les produits, le total et le statut. Chaque bouton propose uniquement l'étape suivante.

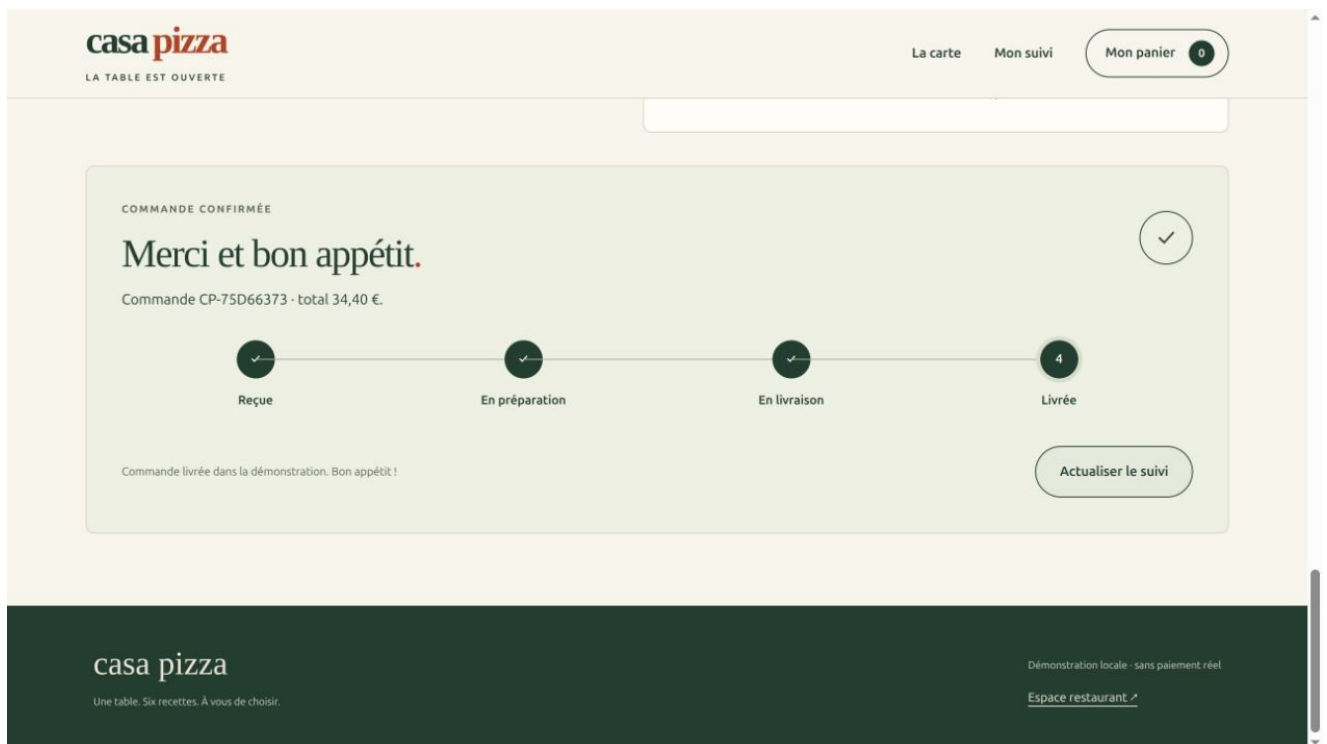
Les étapes autorisées sont Reçue → En préparation → En livraison → Livrée. Le serveur rejette les transitions incohérentes. La recette HTTP vérifie aussi l'accès sans authentification et le mauvais mot de passe.

Gardez cette application sur la boucle locale. Le mot de passe partagé de l'atelier et les sessions en mémoire ne constituent pas une solution de production.

L'avancement est manuel : une personne utilise les boutons de l'espace restaurant. Les statuts ne progressent pas automatiquement avec le temps. La commande reçue ne devient donc pas « En préparation » tant que cette action n'a pas été effectuée.

31 Vérifier le suivi final et le mobile

Revenez à l'onglet client et cliquez sur Actualiser le suivi. Le statut doit correspondre à celui du restaurant.



Sur la commande de référence, les quatre étapes sont vertes après le passage à Livrée.

1. Comparez le numéro côté client et côté restaurant.
2. Vérifiez que Livrée est atteinte après les trois transitions.
3. Répétez une commande dans une fenêtre de 390 pixels de largeur ou sur un téléphone pouvant accéder à un environnement local préparé par le formateur.

Résultat attendu. Le suivi reflète le serveur. À 390 pixels, les cartes, les champs et les boutons restent utilisables sans défilement horizontal.

La référence a été testée dans un navigateur réglé à 390 pixels. Une commande mobile distincte, composée d'une Reine, a été reçue pour 15,40 euros. Le numéro et le panier diffèrent donc du parcours desktop.

32 Observer une commande sur mobile

La vérification mobile porte sur une largeur réelle de 390 pixels. Cette capture montre une autre commande que le scénario desktop.



Une Reine et la livraison donnent 15,40 euros. Les étapes se répartissent sur deux lignes.

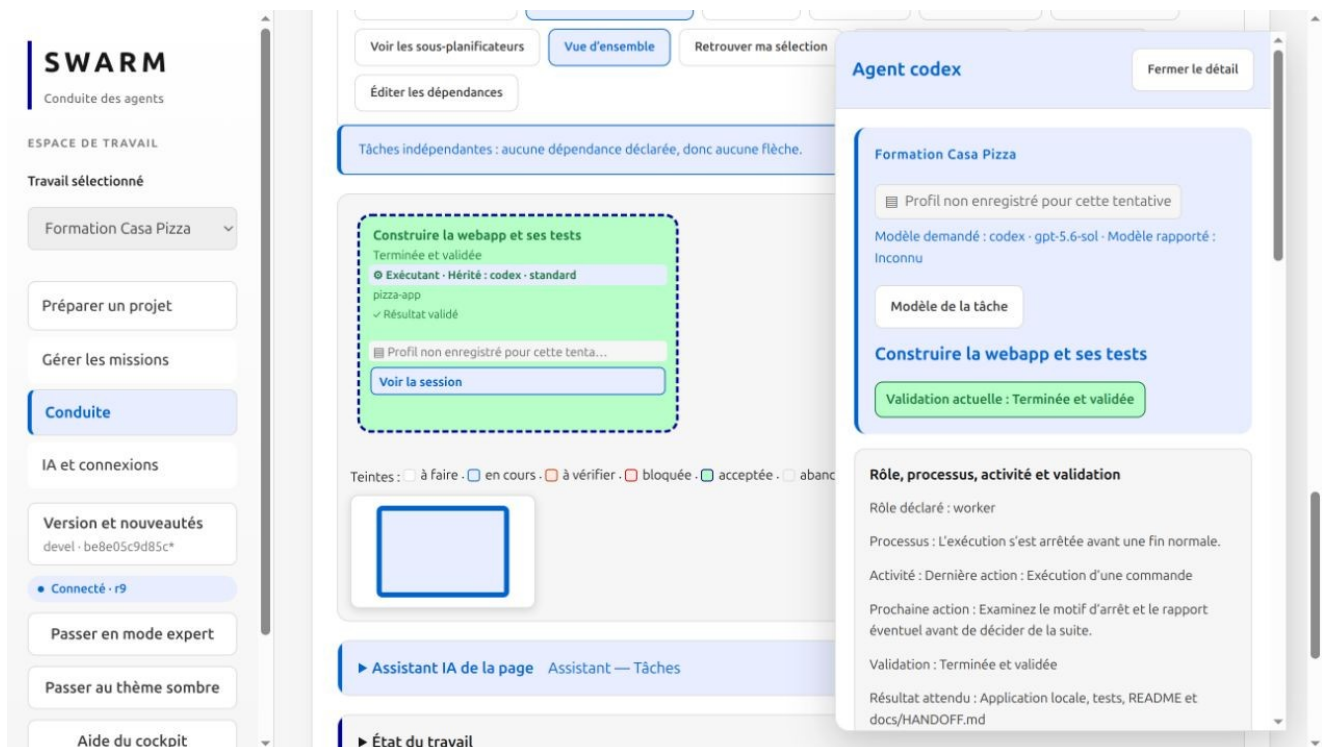
1. Ajoutez une Reine dans un panier neuf.
2. Renseignez les coordonnées fictives et confirmez.
3. Vérifiez le numéro, le total et le bouton Actualiser le suivi sans défilement horizontal.

Résultat attendu. La confirmation reste lisible et utilisable sur un écran étroit.

33 Contrôler avant de valider

La référence livrée passe onze tests : quatre tests applicatifs et sept tests HTTP utilisant un vrai serveur local, dont un redémarrage sur la même base SQLite.

```
python3 -W error::ResourceWarning -m unittest -v
```



Le résultat est accepté après recette. Le détail conserve aussi le fait que le processus agent s'est arrêté avant une fin normale.

1. Dans le dossier du code, exécutez la commande et vérifiez onze tests réussis.
2. Rejouez catalogue, erreur, commande, restaurant et suivi dans le navigateur.
3. Consignez les résultats et les limites dans un rapport de recette.
4. Faites examiner ce rapport avant d'accepter le résultat dans Swarm.

Dans l'atelier observé, le formateur a ajouté les tests HTTP, corrigé la fermeture des connexions SQLite et l'effacement d'une ancienne erreur de panier. Il a soumis docs/FORMATEUR.md, enregistré une gate avec les empreintes des fichiers et accepté le livrable par les opérations publiques Swarm.

Ce contrôle est une recette par le formateur de cette session, pas une revue indépendante. L'acceptation du résultat ne transforme pas l'exécution interrompue en fin normale et ne configure pas une hiérarchie autonome.

Résultat attendu. Vous pouvez justifier la validation par des tests, un parcours réel et un rapport, même lorsque l'historique contient des tentatives en échec.

Après confirmation dans la référence actuelle, le panier est vidé et le suivi reste disponible dans l'onglet. La capture Swarm ci-dessus conserve l'acceptation historique ; le rapport REFONTE.md décrit séparément la recette de la révision affichée.

34 Diagnostiquer Failed to fetch

Ce message signifie que le navigateur n'a pas obtenu la réponse attendue. Dans l'incident observé pendant l'atelier, le serveur Casa Pizza était arrêté.

1. Vérifier le terminal qui exécute app.py. Gardez-le ouvert pendant le parcours.
2. Ouvrir `http://127.0.0.1:18841/api/catalog`. Si la page ne répond pas, vérifier serveur et port avant le formulaire.
3. Relancer depuis le dossier qui contient app.py avec la même base SQLite si vous voulez conserver les commandes.
4. Actualiser la page client, vérifier le panier et confirmer une seule fois. Une nouvelle commande doit afficher un nouveau numéro.

```
python3 app.py
curl -i http://127.0.0.1:18841/api/catalog
```

La référence actuelle affiche Serveur local inaccessible avec une consigne de vérification du terminal. Ce message ne redémarre pas le serveur : l'apprenant doit le relancer.

Si le catalogue répond mais pas la commande

Ouvrez les outils développeur du navigateur, onglet Réseau, puis reproduisez une seule fois. Repérez POST /api/orders. Notez le statut HTTP, la réponse et l'heure ; cherchez l'erreur correspondante dans le terminal serveur. Un refus métier JSON, par exemple le minimum, est différent d'une absence de réponse réseau.

Ouvrez le site via le serveur local, pas directement le fichier HTML. 127.0.0.1 désigne l'ordinateur du navigateur : sur un autre poste, cette adresse ne pointe pas vers le serveur du formateur. Ne concluez pas à un problème CORS sans examiner la requête.

Pour partager un diagnostic, copiez le message, la route, le statut et les étapes. Retirez jetons, cookies, entêtes d'authentification et coordonnées. N'envoyez pas un export réseau complet non relu.

35 Refaire et expliquer le parcours

Avant de terminer, expliquez à un autre débutant comment vous avez obtenu et vérifié le produit.

Exercices et réponses attendues

1. Une Reine seule respecte-t-elle le minimum ? Oui : 12,50 euros de produits, soit 15,40 euros avec livraison.
2. Une Margherita seule suffit-elle ? Non : 9,50 euros de produits, même si le total avec livraison atteint 12,40 euros.
3. Le texte de l'agent prouve-t-il que tout fonctionne ? Non : il faut lire les tests exécutés et rejouer le parcours.
4. À quoi sert Git ici ? À préparer un dépôt de travail et conserver les changements. Le kit ne pousse rien sur un dépôt distant.
5. Le bouton vert signifie-t-il que l'agent a terminé normalement ? Il indique la validation du résultat. Le statut du processus et l'historique doivent être lus séparément.

Avant de fermer

1. Gardez le code, le rapport et les résultats de tests.
2. Arrêtez Casa Pizza avec Ctrl+C dans son terminal.
3. Depuis le dossier Swarm, arrêtez uniquement le serveur de votre atelier.

```
./swarm.sh stop --root "$HOME/projets/pizza-atelier"
```

Si vous aviez choisi 18842, ajoutez --address 127.0.0.1:18842. Ne supprimez ni la base ni l'historique pour résoudre un simple problème de port.

Pour le formateur

Le kit fournit atelier_swarm.py, le code de référence et les captures utilisées ici. Il exclut les clés, les sessions Swarm, les bases de commandes et le dossier Git. Les rapports du producteur sont conservés ; docs/FORMATEUR.md décrit la recette finale et ses limites.

Une prochaine amélioration peut porter sur un seul comportement, par exemple une information sur les allergènes. Écrivez ses critères, lancez une tâche limitée, puis refaites la recette avant validation.

Résultat attendu. Vous savez refaire l'atelier et expliquer pourquoi une livraison vérifiée est différente d'une simple génération de code.

Pause et arrêt

La pause de mission retient de nouveaux départs ; elle ne tue pas les tentatives déjà actives. Arrêter une tentative vise un processus précis. Fermer un onglet ne coupe pas les agents. Avant de fermer l'atelier, consultez les tentatives actives et choisissez l'action adaptée ; arrêter le serveur du cockpit n'équivaut pas à valider les résultats.

36 Périmètre et preuves de cet atelier

La référence permet de tester une application locale. Les capacités avancées décrites doivent être distinguées de celles réellement exécutées dans l'atelier.

Sujet	Portée constatée
Application	Catalogue, panier, commande, SQLite, suivi et progression manuelle restaurant
Vérification	11 tests sur le poste du formateur et parcours navigateur desktop et 390 px
Plan à six tâches	Créé pour la lecture du graphe ; aucune exécution de ces tâches
Politiques	Formulaires et aperçus inspectés ; effets non confirmés
Hors atelier	Paiement réel, logistique, progression temporelle automatique, publication et exploitation en production
Non démontré	Mission hiérarchique autonome, revue indépendante originale et audit complet d'accessibilité

Sources pour vérifier les mécanismes

Référence produit : dépôt Swarm, <https://github.com/moOogly/swarm>. Documents du dépôt : docs/en/USER-GUIDE.md et docs/en/ENGINE-ACCEPTANCE.md. Contrat : tools/agent-workflows/CONTRACT.md. Les politiques et reçus sont décrits dans validation_policy_config.go et automatic_validation.go.

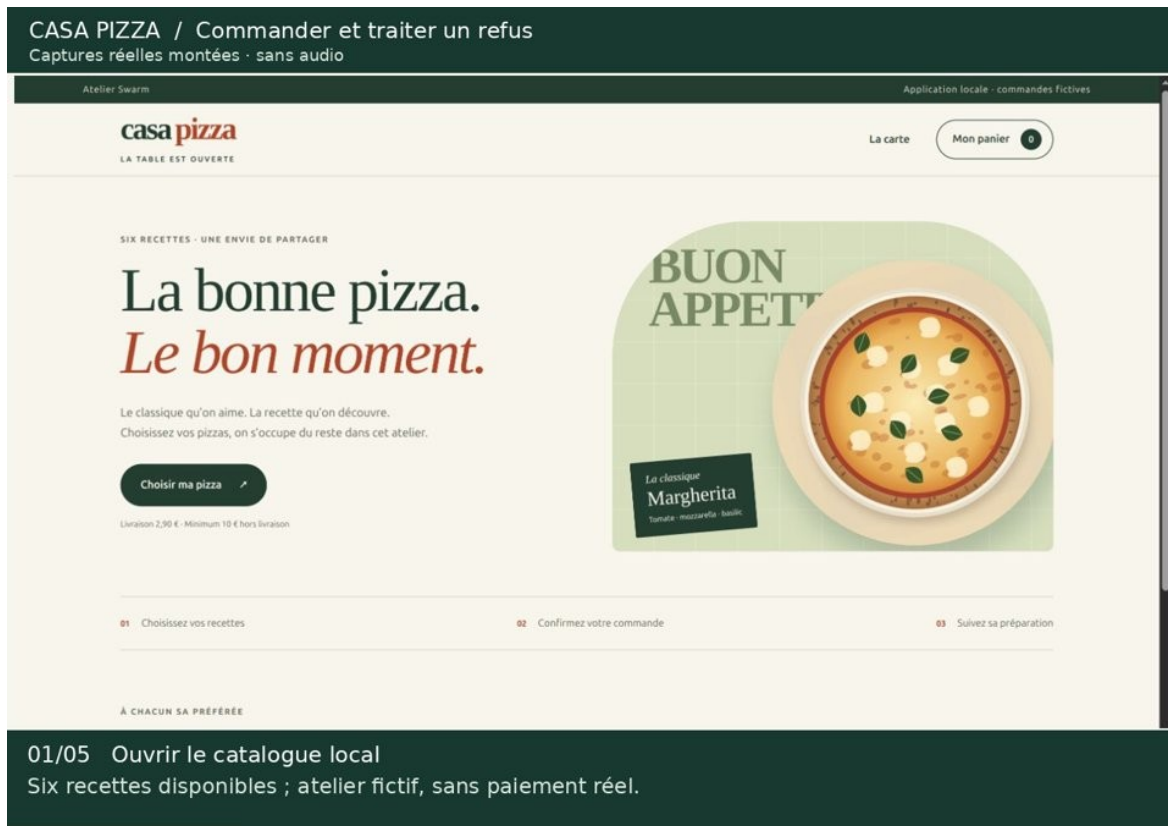
Référence atelier : projet-pizza/docs/FORMATEUR.md, docs/REFONTE.md, tests/test_app.py et tests/test_http.py. Les captures du 9 octobre 2026 montrent une version de développement ; les libellés peuvent évoluer. Aucun résultat de test n'est extrapolé à une autre installation.

Le kit exclut clés, sessions privées, bases de commandes et historique Git. Il ne réalise aucune publication et n'installe aucun service externe. Les rapports historiques du producteur restent en français et distinguent les contrôles non exécutés de la recette finale.

37 Tutoriels animés pour refaire le parcours

Ce complément conserve les 36 chapitres précédents. Ouvrez le lecteur hors ligne, choisissez un module et reproduisez une étape dans votre atelier avant de regarder la suite.

Ouvrir le lecteur pas à pas et les vidéos



Légendes intégrées aux vidéos. Aucun son ni lancement automatique.

Du besoin au graphe · 24 s · chapitres 8–11 et 17–20

Préparer les contrôles Swarm · 24 s · chapitres 21–26

Commander et traiter un refus · 30 s · chapitres 27–30

Du restaurant au suivi client · 42 s · chapitres 31–34

Extraire entièrement le ZIP avant ouverture. Les MP4 peuvent aussi être ouverts directement dans tutoriels/media. Le GIF est un aperçu réduit ; le mode pas à pas garde chaque image à l'écran et la transcription fournit les explications.

38 Exercices corrigés et portée des images

Mettre en pause, répondre, puis vérifier. Les corrigés sont également accessibles dans le lecteur.

1 Du besoin au graphe

Quelle tâche doit précéder les tests ? Le graphe prouve-t-il une exécution ?

Commande et suivi précède Tests automatiques. Non : les six tâches sont non exécutées, sans agent actif. Le plan a été créé par le script pédagogique, séparément du besoin enregistré.

2 Préparer les contrôles Swarm

Une commande réussie suffit-elle à déclarer la mission terminée ?

Non. Elle apporte une preuve sur les fichiers et critères liés au candidat. La recette visuelle, la revue et les conditions d'acceptation restent distinctes. Dans cet enregistrement, aucune politique n'a été appliquée et aucune tâche n'a été acceptée.

3 Commander et traiter un refus

Pourquoi 12,40 euros livraison comprise ne suffisent-ils pas pour une pizza ?

Le minimum s'applique aux pizzas seules : 9,50 euros reste inférieur à 10 euros. La livraison de 2,90 euros ne compte pas dans ce minimum.

4 Du restaurant au suivi client

Qui a fait avancer la commande ? Comment vérifier que c'est la même ?

Le formateur, via les boutons du restaurant. Comparer numéro, contenu et total entre restaurant et client. Le parcours ne démontre ni livraison réelle ni agent autonome.

Ce qui a réellement été enregistré

Le 9 octobre 2026, des actions ont été effectuées dans un cockpit et une application locale isolés. Les captures successives ont été assemblées en vidéos fixes et GIF avec légendes ; il ne s'agit pas d'une capture vidéo continue. Aucun fournisseur IA n'a été appelé pour cette démonstration.

Le besoin a été enregistré. Le plan pédagogique a été créé séparément par le script public, sans exécution des six tâches. Les politiques de contrôle ont été prévisualisées puis annulées. Le parcours client et les transitions restaurant ont réellement été joués avec des données fictives. Les onze tests de la référence ont réussi ; cela ne prouve pas une exécution autonome de la mission.